AI FitTrack API

An AI-Powered Fitness Tracking & Workout Recommendation Platform

Project Team

- **Elizabeth Mary Roberts** — Team Lead
- Kowsalya N
- Nedha Thasef H
- Rafathunissa S

Overview

AI FitTrack API is a secure REST API service built to help users log workouts, track fitness progress, and receive AI-generated workout recommendations and fitness insights. The backend runs on Node.js and Express.js, with MongoDB as the primary datastore and Mongoose acting as the object-data modeling layer that enforces schema rules.

Security sits at the core of the platform: JSON Web Tokens (JWT) provide stateless session handling and protect restricted routes, while bcrypt keeps stored passwords irreversibly hashed. Behind the scenes, the service manages AI-driven workout recommendations and fitness-insight generation through Google's Gemini AI. A centralized error-handling layer sanitizes response codes so that raw database failures — duplicate keys, validation errors, malformed IDs — are never exposed to clients, keeping the system stable and predictable.

Case Study: A Day in Arjun's Work

Background

Arjun is a fitness enthusiast juggling a busy work schedule. He wants to stay consistent with his workouts, but doesn't always know what routine suits his goals, and he rarely has time to analyze his own progress.

The Challenges He Faces

- Keeping a consistent, organized log of every workout session is tedious to do by hand.
- He isn't sure which workout category or intensity best fits his current fitness level and goals.
- Spotting trends in his own calories burned, duration, and consistency over time takes manual effort.
- There's no easy way to get personalized guidance without hiring a trainer.

How AI FitTrack Helps

AI FitTrack's backend acts as a centralized, secured API layer that separates core business logic from whatever front-end eventually consumes it. Internally, it runs authentication checks and hands off requests to dedicated controllers for account management, workout logging, and AI-based recommendation generation.

Who Uses What

- **Registered Users:** Create an account, log in, log and manage their own workouts, and request AI-generated workout recommendations and fitness insights.
- **Public/Anonymous Visitors:** Can access only the public registration and login endpoints; all fitness and AI features require an authenticated session.

Results

Pushing workout-tracking logic into a dependable backend cuts down on manual effort. Automated schema checks keep workout data consistent, JWT-based tokens ensure only the account owner can access their own data, and the Gemini AI integration turns raw workout history into actionable recommendations.

1. Software Requirements

- Operating System — Windows 10/11, macOS, or Linux; the stack runs cross-platform.
- Node.js v18 or newer — the JavaScript runtime that executes the server logic and routing.
- npm v9 or newer — manages the project's package dependencies.
- Express.js v4 or v5 — the lightweight web framework used to build RESTful endpoints.
- MongoDB with Mongoose ODM — the NoSQL document store holding users and workout records.
- Google Gemini API (@google/genai) — powers AI-generated workout recommendations and fitness insights.
- Postman or Thunder Client — used to exercise the API and confirm schema and route-guard behavior.
- A code editor — Visual Studio Code or an equivalent IDE.

2. Hardware Requirements

- Processor — Intel Core i5 (8th generation or later) or an equivalent AMD Ryzen 5.
- RAM — 8 GB minimum; 16 GB recommended when running MongoDB alongside the dev server.
- Storage — at least 1 GB of free disk space.

Epic 1 — System Architecture

AI FitTrack exposes a set of well-defined endpoints managed by a layered server design. Every incoming request passes through several filtering stages before it ever touches the data layer.

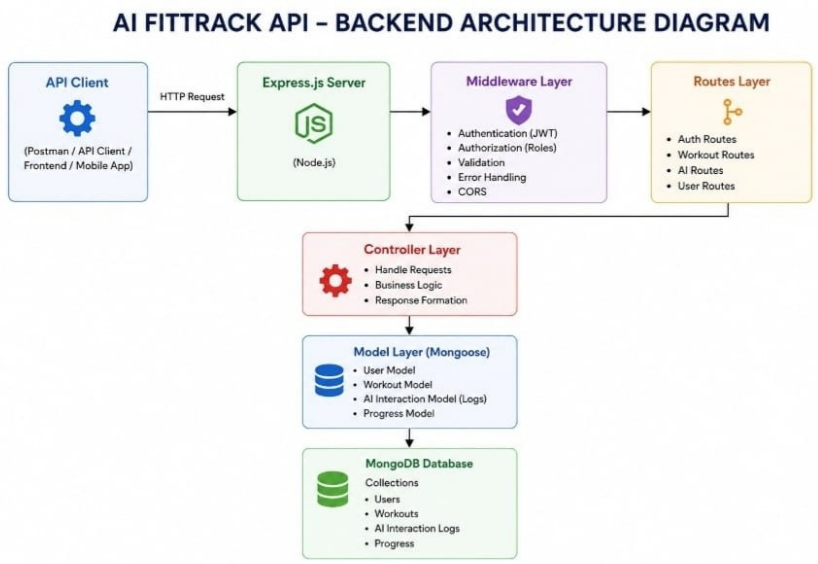


Figure 1: AI FitTrack API — backend architecture diagram

Layer-by-Layer Breakdown

- **Client Interface:** Tools such as Postman send requests to specific routes and receive JSON back.
- **Express Gateway (app.js):** Listens on the network port, maps HTTP verbs (GET, POST, PUT, DELETE), parses JSON bodies, enables CORS, and logs requests via Morgan.
- **Authentication Middleware (auth.js):** Pulls the JWT out of the Authorization header, verifies it against the JWT_SECRET, and attaches the authenticated user to the request before allowing it through.
- **Controllers:** authController, workoutController, and aiController each own a narrow slice of domain logic — account management, workout CRUD, and AI generation respectively.

- **Data Access Layer:** Mongoose converts application-level operations into efficient native MongoDB queries against the User and Workout collections.
- **AI Integration (geminiService.js):** The AI controller hands off workout-recommendation and fitness-insight generation to Google's Gemini SDK.
- **Centralized Error Handler:** Normalizes CastErrors, duplicate-key errors, and Mongoose validation errors into a consistent JSON error shape.

Entity Model

The database is composed of two core MongoDB collections, linked by a logical reference, covering authentication and workout tracking. The full planned data model — including workout categories, exercises, AI interaction logs, and progress tracking — is shown in the ER diagram below.

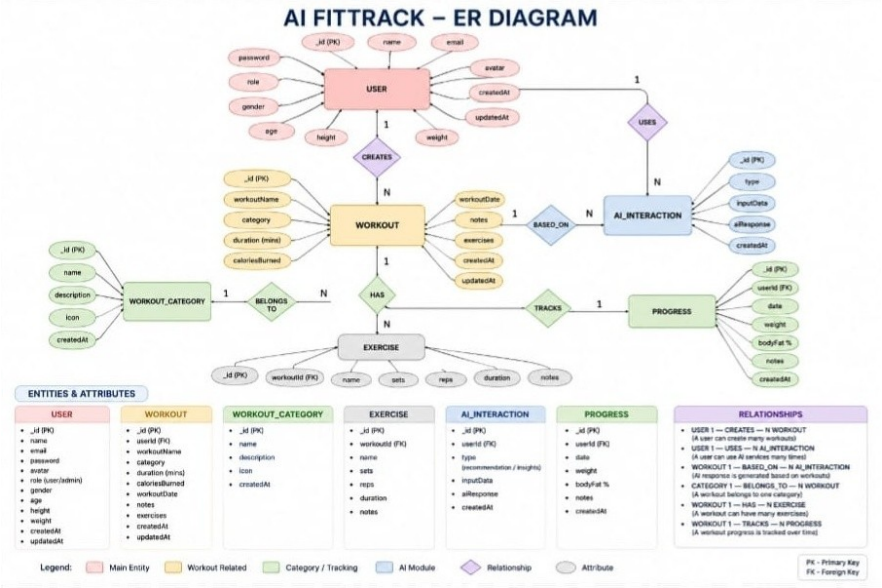


Figure 2: AI FitTrack — Entity-Relationship diagram

User Collection

- _id — primary key (ObjectId)
- name — String, required
- email — String, required, unique, lowercase, validated against a standard email pattern
- password — String, required, minimum 6 characters, hashed with bcrypt before storage
- createdAt / updatedAt — automatic timestamps

Workout Collection

- _id — primary key (ObjectId)
- workoutName — String, required
- category — String, required; one of Cardio, Strength Training, Yoga, Running, Cycling, Walking
- duration — Number, required, minimum 1 (minutes)
- caloriesBurned — Number
- createdAt / updatedAt — automatic timestamps

How the Pieces Connect

- **User → Workout (one-to-many):** A single registered user can log and manage many workout entries.
- **User/Workout → AI Insight (logical):** The AI controller reads a user's workout history to generate personalized recommendations and insights via Gemini.

Highlights

- **JWT-based Access Control:** Token-driven authentication locks down private endpoints.
- **AI-Powered Recommendations:** Google Gemini AI generates personalized workout recommendations and fitness insights.
- **Full Workout Logging:** Workout records can be created, read, updated, and deleted.
- **Category Enforcement:** Workouts are validated against a fixed set of categories (Cardio, Strength Training, Yoga, Running, Cycling, Walking).
- **Encrypted Credentials:** Passwords are hashed with bcrypt before they're ever stored.
- **Unified Error Handling:** A centralized handler standardizes API error responses for easier debugging.
- **Schema Enforcement:** Mongoose validation guarantees data integrity ahead of persistence.
- **RESTful Design:** Modular Express routes (auth, workout, AI) keep backend services scalable.

Roles and What They Can Do

Authenticated User

- Register an account and log in to receive a JWT.
- View and manage their own profile.
- Log, update, and remove their own workout entries.
- Request AI-generated workout recommendations.
- Request AI-generated fitness insights based on their workout history.

Public / Anonymous User

- Can access the registration and login endpoints only.
- Cannot view, create, edit, or delete workout records.
- No access to any protected API endpoint.

MVC Design Pattern

The codebase is organized around the Model-View-Controller pattern, which keeps each layer's responsibilities cleanly separated:

Model Layer — Data Blueprint

Defines strict Mongoose schemas (User, Workout) that map application fields onto raw MongoDB documents, applying field rules and defaults ahead of validation.

Controller Layer — Request Orchestration

Acts as the system's coordinator: authController, workoutController, and aiController receive requests from the routing layer, check that the request is well-formed, hand work off to the right model or service, and package the outcome into a standard JSON response.

View Layer — Routing Interface

Since this is a headless backend with no rendered UI, the “view” here is really the API routing layer — authRoutes, workoutRoutes, and aiRoutes each map incoming requests on a given path directly to the matching controller handler.

Epic 2 — Setting Up the Project

Creating the Project Folder

- Create a new folder using your project's name (e.g. AI-FitTrack-API).
- Inside it, add a subfolder called src.

- Open the folder in VS Code.

Initializing the Server (npm init)

From the project root, open a terminal in VS Code and run:

```
npm init -y
npm install express mongoose dotenv cors jsonwebtoken bcryptjs morgan
@google/genai
```

Folder and File Layout

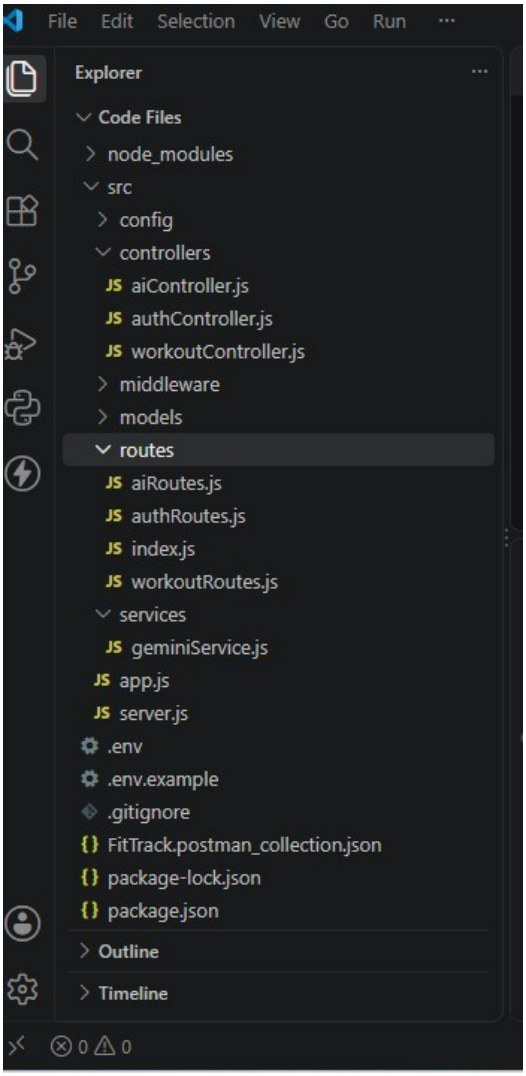


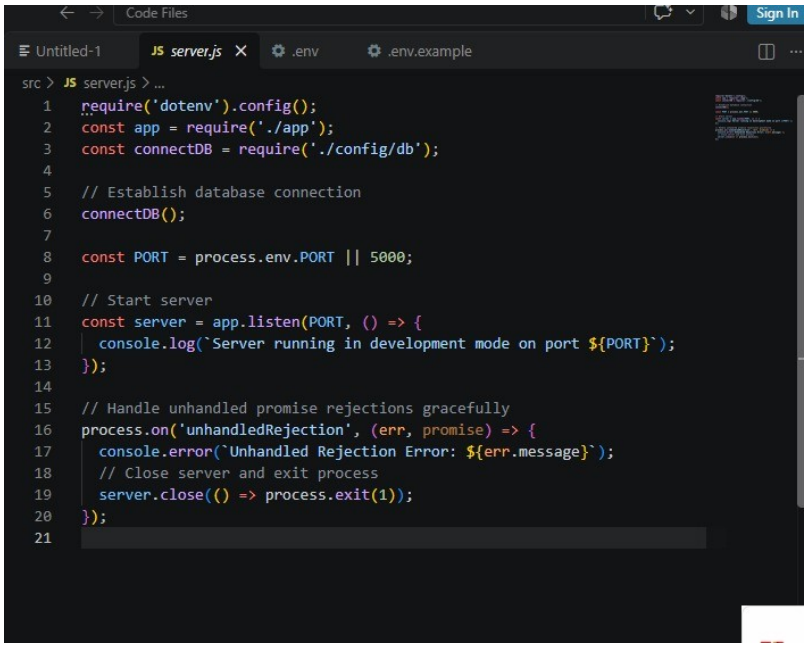
Figure 3: Full project directory structure once all modules are in place

Epic 3 — Building the Backend

Core Script Implementations

1. Entry Point (src/server.js)

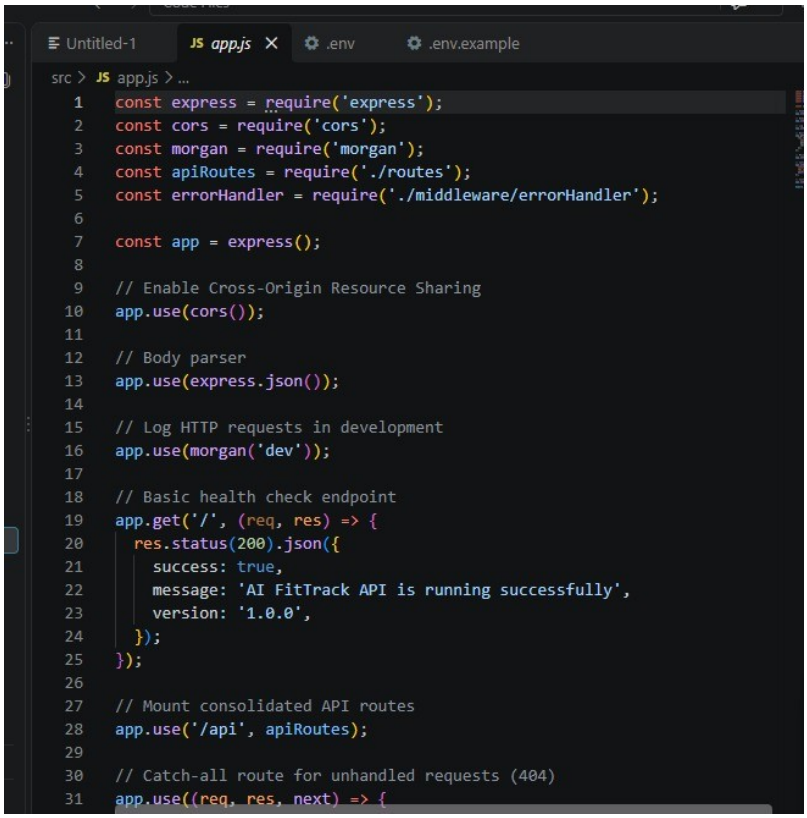
This is the application's launch point — it loads environment variables, opens the database connection, and starts listening on the configured port.



src/server.js

2. Express App Setup (src/app.js)

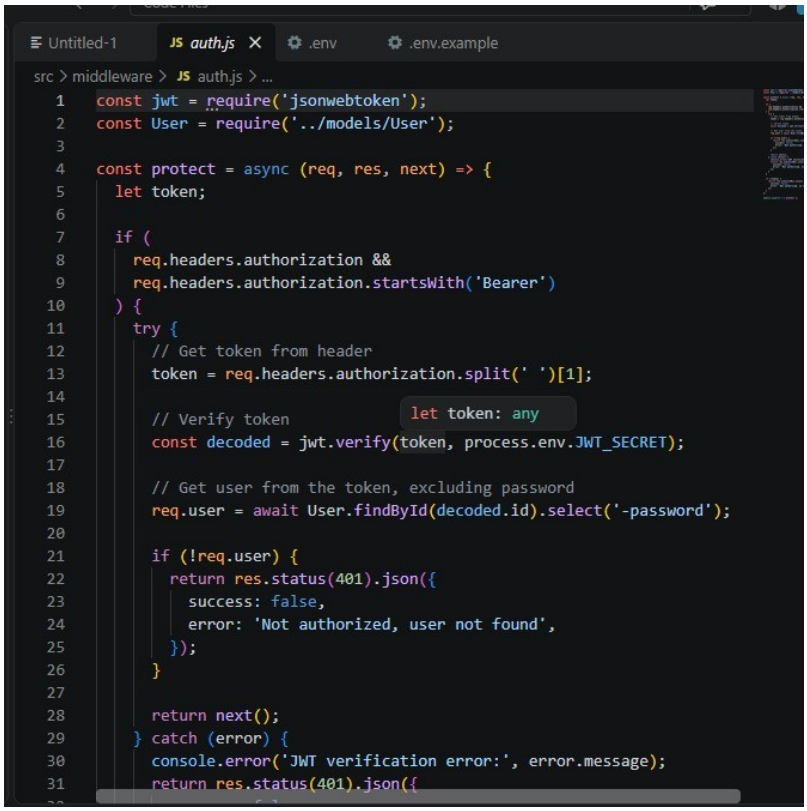
Configures the core Express instance: CORS, JSON body parsing, request logging via Morgan, the health-check route, and mounting of the consolidated API router.



src/app.js

3. Authentication Middleware (src/middleware/auth.js)

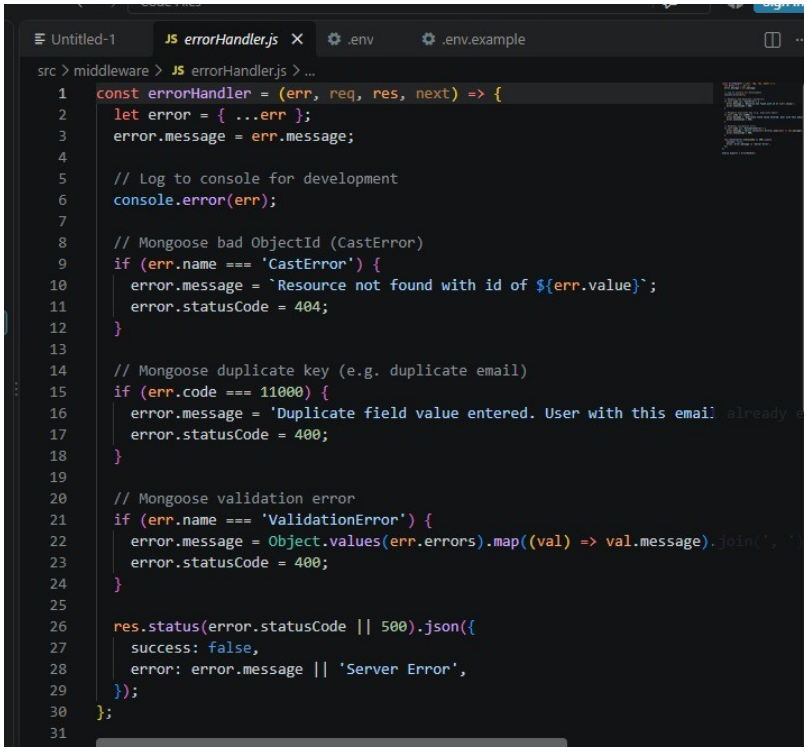
Intercepts incoming requests, pulls the Bearer token from the Authorization header, verifies it against JWT_SECRET, and attaches the authenticated user (minus the password field) to the request before allowing it through.



src/middleware/auth.js

4. Centralized Error Handler (src/middleware/errorHandler.js)

Normalizes every error response the API returns, covering duplicate-key errors, Mongoose validation failures, and malformed ObjectIds, while logging the full error to the console for development.



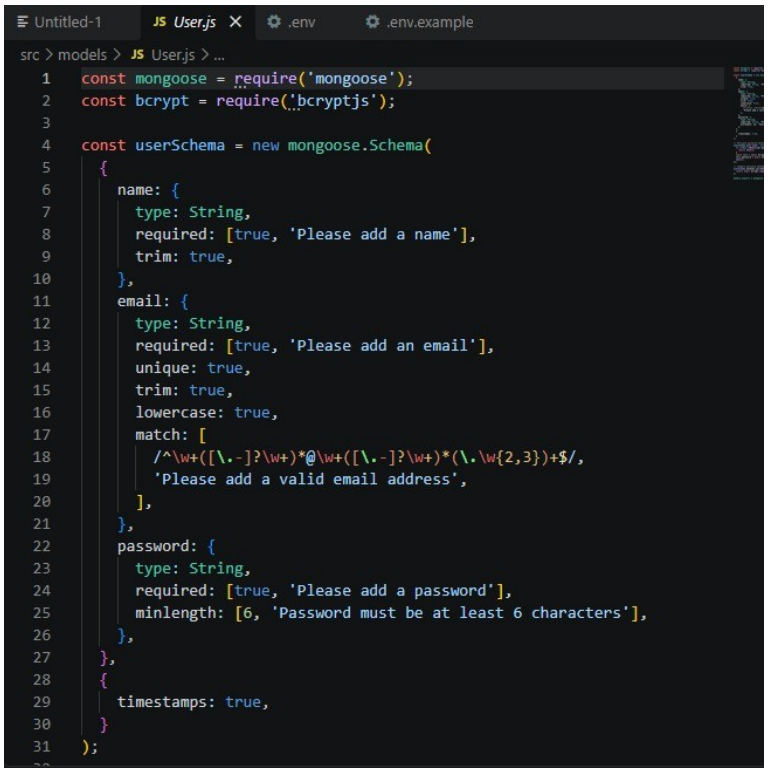
src/middleware/errorHandler.js

Epic 4 — Database Configuration & Routes

Mongoose Schema Definitions

1. User Schema (src/models/User.js)

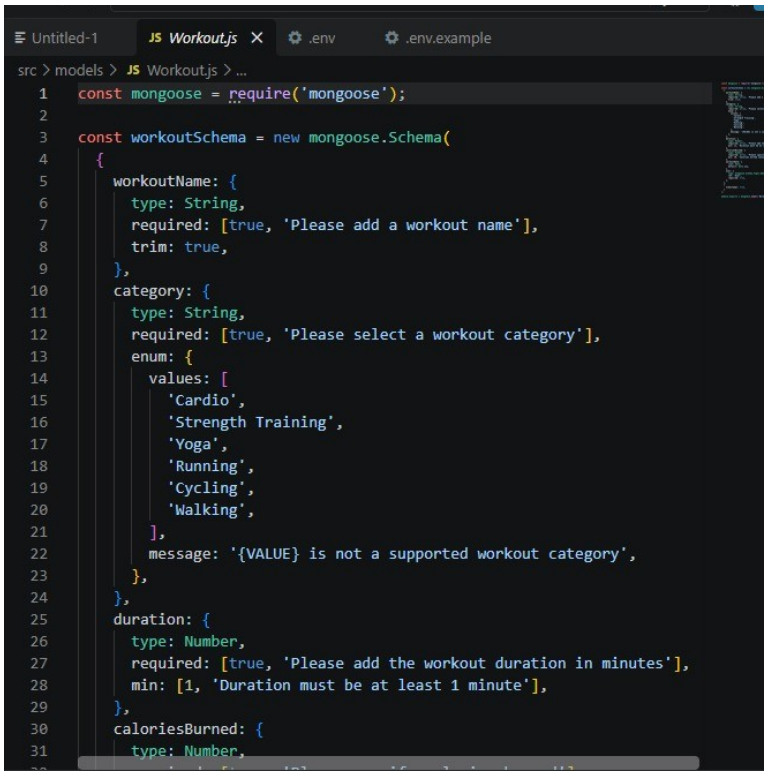
Validates each profile field — name, email format and uniqueness, and a minimum-length password — ahead of persistence.



src/models/User.js

2. Workout Schema (src/models/Workout.js)

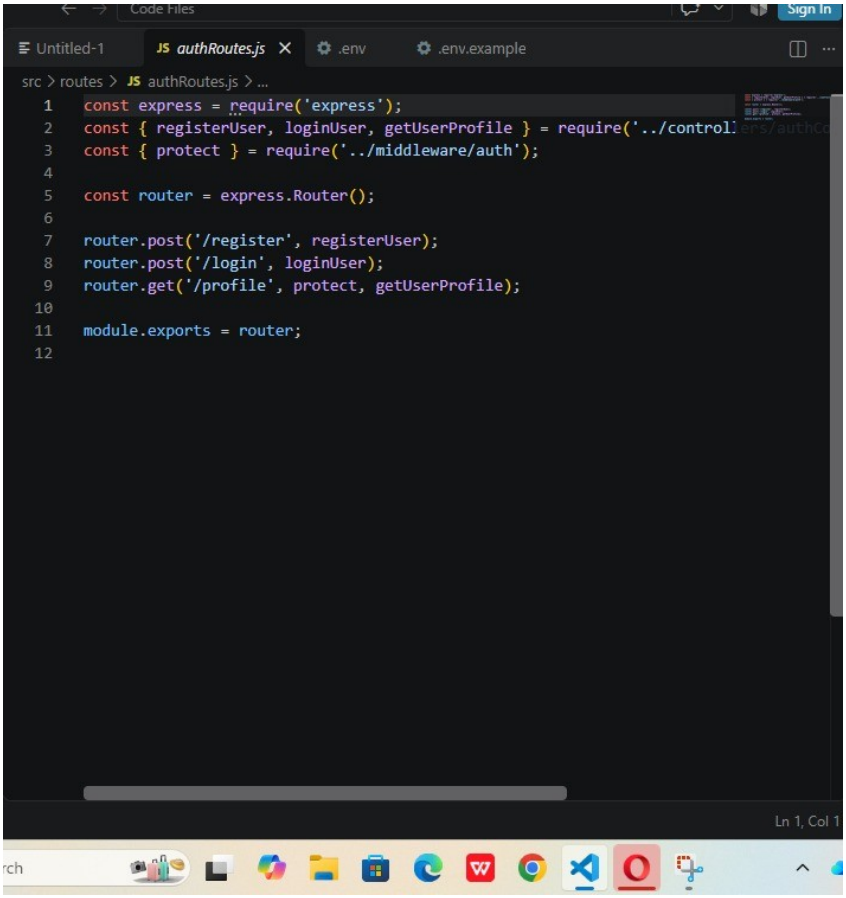
Defines the core workout fields, restricts category to a fixed enum, and enforces a minimum workout duration.



src/models/Workout.js

API Route Definitions

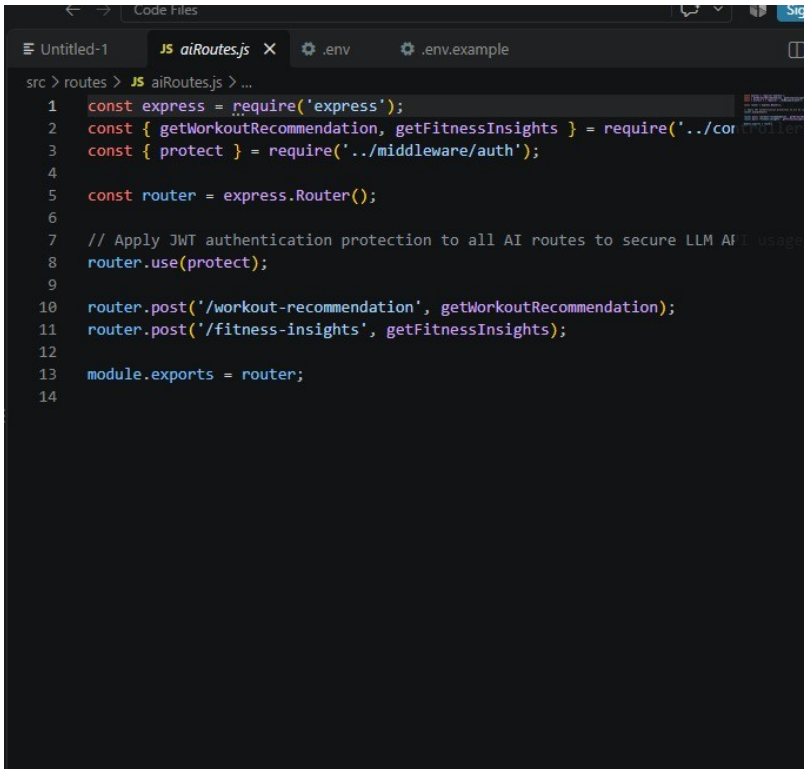
1. Public Authentication Routes (src/routes/authRoutes.js)



src/routes/authRoutes.js

2. Protected AI Routes (src/routes/aiRoutes.js)

The protect middleware is applied to every route in this file, so all AI endpoints require a valid JWT.



src/routes/aiRoutes.js

Epic 5 — Running and Testing the API

Starting the Server

- Install Postman, or the Thunder Client extension for VS Code.
- Launch the development server with the command below.

```
npm run dev
```

Sample Endpoint Walkthrough

The requests below confirm that each endpoint behaves as expected.

1. User Registration

- Route: POST /api/auth/register
- Access: Public

Sample request body:

```
{
  "name": "Pooja",
  "email": "poojadnarayanan@gmail.com",
  "password": "pooja@123"
}
```

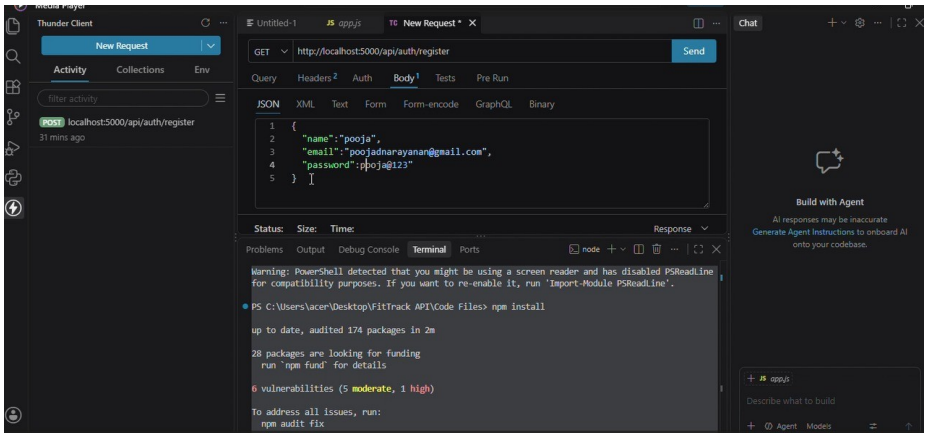


Figure 4: Registration request in Thunder Client

2. User Login

- Route: POST /api/auth/login
- Access: Public

Sample request body:

```
{
  "email": "poojadn@gmail.com",
  "password": "pooja@123"
}
```

Response: a JWT token, to be sent as a Bearer token in the Authorization header for all protected routes.

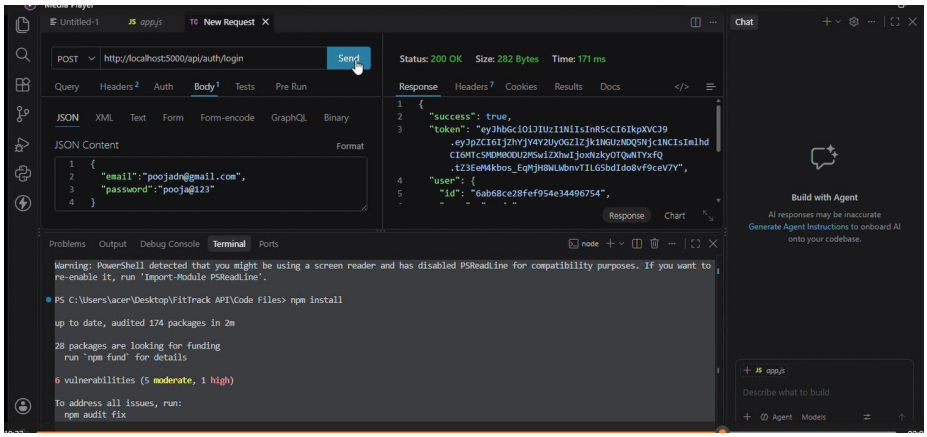


Figure 5: Login response returning a JWT in Thunder Client

3. Get User Profile

- Route: GET /api/auth/profile
- Access: Private (Bearer <JWT Token>)

4. AI Workout Recommendation

- Route: POST /api/ai/workout-recommendation
- Access: Private (Bearer <JWT Token>)

Sample request body:

```
{
  "age": 20,
  "fitnessGoal": "maintain a healthy body weight",
```

```
  "experience": "beginner"
}
```

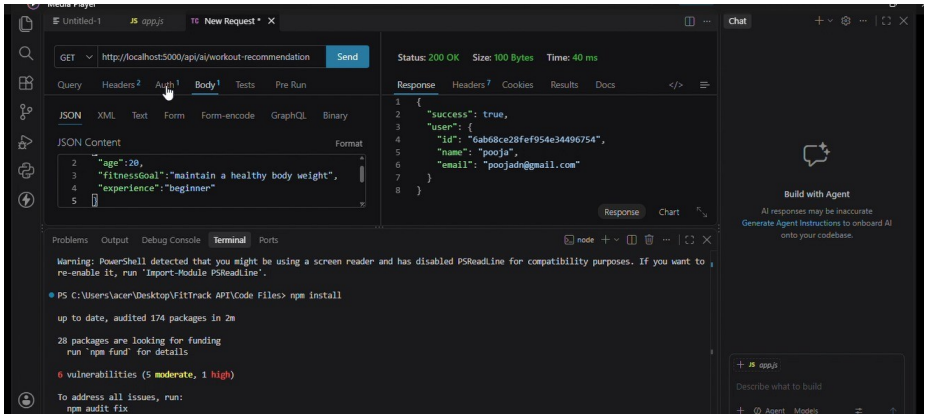


Figure 6: AI-generated workout recommendation response in Thunder Client

5. AI Fitness Insights

- Route: POST /api/ai/fitness-insights
- Access: Private (Bearer <JWT Token>)

Health Check

A simple GET request to the API root confirms the server is running:

```
GET /

{
  "success": true,
  "message": "AI FitTrack API is running successfully",
  "version": "1.0.0"
}
```

Reference Links

API Testing Walkthrough (Video): https://drive.google.com/file/d/1XrBv-GOAYqrwaD5v7IOOzDy83UjbLg3p/view?usp=drivesdk

Project Demo: https://drive.google.com/file/d/1WMp9toT6wjRcXllhet-bXtfFGjw_J5a8/view?usp=drivesdk